

AgentBrain

When and Why Agents Search

A clear explanation of how the brain decides when to call `web_search`, when to wake up at all, and what the ReAct pattern means in practice. Includes the current gap and what needs to be built next.

May 2026 · SDK v0.5.0 · Generated 2026-05-11 20:37

1. The Core Question

The user asked: *"Does the agent search randomly, or does it know when to search?"*

Short answer: **not randomly**. There are two completely separate decisions happening, and understanding them is key to building a precise, cost-efficient autonomous agent.

Decision	Who makes it?	Cost	Question answered
When to wake up	Trigger layer (our Python code, no LLM involved)	Free (timer / lightweight check)	Is it time to check anything?
Whether to search	The LLM (during judgment cycle)	Paid (LLM tokens + search API)	Do I have enough context to make a decision?

2. Decision 1 — When to Wake Up (Triggers)

Triggers are pure Python — no LLM, no API cost. They define *when the brain runs a cycle*, not what it does.

Trigger	When it fires	Best for	Code
IntervalTrigger	Every N seconds (min 10s)	Regular monitoring e.g. every 30 min	IntervalTrigger(seconds=1800)
DailyTrigger	Once per day at a UTC time	Daily reports e.g. 9am market open	DailyTrigger(hour=9, minute=0)
OnChangeTrigger	Only when a perception source reports changed=True	Event-driven e.g. price spike, new commit	OnChangeTrigger(perception, poll_interval=60)

3. Decision 2 — Whether to Search (The ReAct Pattern)

Once awake, the brain calls the LLM. The model uses the **ReAct pattern** (Reason + Act) — alternating between reasoning about what it knows and taking actions to fill gaps.

The loop in plain English:

1. LLM reads: mission + perceptions + memory context
2. LLM reasons: *"Do I have enough information to judge this?"*
- 3a. If YES: output JSON judgment directly — no search needed
- 3b. If NO: call `web_search(query)` or `fetch_url(url)`
4. LLM reads the tool result

- 5. Back to step 2 — repeat up to 8 rounds
- 6. Final output: {should_act, reasoning, summary, actions[]}

What drives the LLM to search:

Driver	Example	LLM behaviour
Mission	"Monitor oil markets"	LLM knows it needs current data — searches every wake-up because its training data is up to date
Perceptions	GitPerception shows new commit with unfamiliar library	LLM searches for CVEs related to that library before judging severity
Memory	Note: "OPEC meeting Thursday"	LLM searches for OPEC outcome on Friday — context from prior cycle
Gap in knowledge	No perceptions configured, tool-only mode	LLM always searches — it has no data without tools

4. The Right Architecture — Two-Phase Intelligence

The current system has a gap: the LLM searches on every trigger cycle, even when nothing changed. The correct model uses two phases:

Phase	What happens	Cost	Example
1 — Detection (cheap)	A lightweight check asks: "Did anything material change?" No LLM. Just a number comparison or RSS feed check.	Near-zero (one HTTP call)	Oil price API: did Brent move more than 3% in the last hour?
2 — Research (expensive)	Only triggered by Phase 1. LLM uses web_search + fetch_url to understand WHY and WHAT to do.	LLM tokens + search API	"Oil dropped 8%. Let me search for the cause before alerting."

How this maps to the existing trigger system:

Phase 1 is an **OnChangeTrigger** wrapping a cheap perception source (a price API that only returns changed=True when the threshold is crossed). Phase 2 is the normal LLM judgment cycle that fires when the trigger activates.

Oil market example — correct setup:

```
brain.add_trigger( OnChangeTrigger( APIPerception(url='https://api.prices.com/brent',
extract='price', threshold=3.0), poll_interval=60, # cheap: checks every 60s ) ) # LLM only
wakes when oil moved >3% -- not every hour
```

5. What Needs to Be Built

The current system is architecturally correct but needs these improvements to become production-grade:

Step	What to build	Why	File
1	Threshold support in APIPerception	Add a threshold= parameter so the perception only reports changed=True when a numeric value moves past a configured amount. e.g. threshold=3.0 means 3% change required to trigger.	brain/perception/api.py
2	Smarter system prompt	Update the system prompt to say: if perceptions already have sufficient context, output the JSON judgment directly without calling any tools. This prevents unnecessary searching when GitPerception or APIPerception already provided complete data.	brain/judgment/engine.py _SYSTEM_PROMPT constant

3	Cost tracking in BrainMemory	Record how many tool calls and LLM tokens were used per cycle. Helps users tune their trigger cadence and identify expensive missions.	brain/memory/store.py
4	Pre-built detection perceptions	PricePerception(ticker, threshold%) — wraps Yahoo Finance or Alpha Vantage. NewsTitlePerception(query) — polls RSS feeds, fires only on new articles.	brain/perception/price.py brain/perception/news.py